DEVOPS & CLOUD [SWE] · SCALER ACADEMY

$ cat notes.md

DevOps & Cloud [SWE] — Class 4 Notes

LECTURE TITLE	**Networking Fundamentals**
DATE/TIME	**19 August 2026, 5:15 PM**
DURATION	**120 minutes**
INSTRUCTOR	**Nensi Ravaliya**
BIG IDEA	**Every website you open is a package delivery — an address (IP), a street (subnet), a phonebook (DNS), and a courier that either signs for it (TCP) or doesn't (UDP).**
SOURCES	**this class's transcript + the instructor's networking reference repos, reorganised in §18: Networking · How-DHCP-Works · Subnetting · IP-quest · IPFIX-NETFLOW-NTP · Network-Troubleshooting · OSI-Network-devices**

WHAT'S INSIDE — 20 SECTIONS

00 ⚡ The 60-Second Version

01 🗓️ Housekeeping & Homework Review

02 🟫 Live Review — The While-Loop Script

03 📔 Her Handmade Cheat-Sheet Book

04 🌐 Networking Kickoff

05 🔢 IP Addresses — The Basics

06 🧱 IP Classes, Subnet Masks & Broadcast

07 🔷 What Happens When You Open a Website

08 🔷 Changing Your IP (and How Not To Lock Yourself Out)

09 📡 DHCP — The DORA Handshake

10 🧱 Subnetting Refresher

11 🔀 TCP vs UDP, and the OSI Layers

12 🎮 Ports Quiz

13 🌐 Hands-On — The Networking Commands

14 📝 Homework

15 👋 Wrap-Up

16 📔 Cheat-Sheet — Commands & Concepts

17 🔷 The Interview Question Set (from IP-quest)

18 🌐 Reference Material — Her Networking Repos

19 🗒️ Notes on the Transcript Itself

Rebuilt from the raw class transcript — nothing cut, just organised, de-noised and made readable. `theme: Grape Soda`

00 ⚡ The 60-Second Version

1. **An IP address is a unique address for a device.** 32 bits, written as 4 numbers of 8 bits each: `192.168.1.50`.
2. **A subnet mask does exactly one job:** it tells you which bits are the **network** and which are the **host**. That's it. Nothing else.
3. **CIDR (`/24` , `/26`) exists because the old A/B/C classes ran out of addresses.** It lets you size a network to what you actually need.
4. **DHCP gives your device its IP, automatically, in 4 steps: DORA** — **D**iscover → **O**ffer → **R**equest → **A**cknowledge.
5. **TCP acknowledges every packet (reliable, slower). UDP doesn't (fast, lossy).** HTTP/HTTPS ride on TCP.
6. **HTTPS = HTTP + a TLS certificate** doing an encrypted handshake.
7. **Never run `sudo` with an IP-changing command.** You can permanently change your IP and lock yourself out of the server.
8. **The interview questions are all subnet maths**, not diagrams. "How many host bits in `255.255.252.0`?" → 10. See §17.

◆ ◆ ◆

01 🗒️ Housekeeping & Homework Review

The mic saga

The class opened with audio trouble — she couldn't find her second microphone, there was background noise, and a lot of *"Am I audible? Clearly? It's fine, huh?"* Eventually: *"Okay guys, can I have your attention?"*

Last class, and the PR homework

Recap: Class 3 covered **variables, runtime input, and conditions**.

The homework was a real open-source workflow:

```
1. FORK the DevOps Heroes repo
2. Run one of the scripts in it
3. Make some modifications
4. Raise a PULL REQUEST back to the repo
5. She reviews it
```

🎤 SAID IN CLASS

The scoreboard: *"I got 30 PRs from 2 classes. Can you believe it — from two batches?"*

And the nudge: *"Other people who have not raised a PR yet — they have to."*

😄 Batch confusion

She lost track of which batch had covered what:

😄 CLASS MOMENT

"Yesterday evening batch or yesterday morning batch, you are at? Morning. Then I guess we covered until condition and input."

A student corrected her: *"We were sort of understanding what a variable is, and then we started with functions — at least I ran the file, but we did not move forward."*

Later, again: *"I guess there were two tasks, right? But I forget what the second task was… Next time I will make sure I note which task I gave to which batch."*

◆ ◆ ◆

02 📜 Live Review — The While-Loop Script

She ran the interactive while-loop script from Class 3 (heard as *"value.sh"* and *"wildhook.sh"* at different points in the recording — same file, ASR noise; it's `while_loop.sh`).

First problem, unsurprisingly:

```
permission denied
```

🎤 SAID IN CLASS

"Permission denied — what will you do? Then we `chmod +x`. Okay, then the file. Now: `./value.sh` — and I can see 'enter the number'."

🎯 The good question: is `true` a variable or a keyword?

```
while true; do
```

🎯 INTERVIEW ALERT

She put it to the room: *"`while true` — true means what, do you think? Is it a variable, or a predefined word we have in Linux?"*

The room split. *"How many of you are thinking it's a variable? Predefined. Nice, good."*

✅ **Answer: predefined.** `true` is a built-in that always succeeds — you don't declare it. Which is why `while true` loops forever until something `break` s it.

The walkthrough

Input	What the script does
`q`	prints *"Exiting the loop"* → `break` — leaves entirely
`7`	prints *"You entered: 7"*, then **asks again**
`9`	prints 9, **asks again**
`10`	prints 10, **asks again**
letters / anything non-numeric	prints *"Invalid input"* → `continue` — skips back to the prompt
`Ctrl + C`	hard-kills the script from outside

✅ LOCK THIS IN

The point she hammered: *"After I entered 9, again it is asking me for a number. Why? Because of `continue` — we did not break it."*

`break` **leaves the loop.** `continue` **goes round again.** That's the whole difference, and it's why only `q` ends the program.

📝 The assignment (5 minutes, in class)

📝 **Run the script, create `while_loop.md`, and explain in 3–5 lines what you learned from it.** Then push it and ping the repo link in chat.

🎤 *"At least 10 GitHub repos — I need 10 repos right now. You have 5 minutes."*

😄 Live review of what students pushed

She opened student repos on screen:

What she found	Her reaction
A file showing **0 bytes**	*"See — 0 bytes. Just add it… Maybe it is not committed yet. Do the `git status` and check."*
A good writeup	*"Nice explanation by you. Good work."*

What she found	Her reaction
Another one	*"So it's not Kavya — it is the pronunciation. Kavya. Good, good explanation."*
One student pushing back	*"I feel like I'm very special"* 😄
Someone who tried and struggled	*"At least she tried. Good work."*

✅ LOCK THIS IN

Steal this debugging habit: `git status` **before you claim a push worked.** A 0-byte file on GitHub almost always means you committed nothing — the classic "I saved the file but never staged it."

◆ ◆ ◆

03 📔 Her Handmade Cheat-Sheet Book

Between topics, she pulled out her own revision notes from when she was learning.

🧭 **The story.** *"I teach you from my book. When I was learning, I created something like this. It's like a cheat sheet."*

What's in it: Linux, networking, GitHub, Docker, Kubernetes, Ansible, and *"maybe Terraform"* — one topic per page.

Networking? *"I covered like maybe 2 to 3 pages. Only 3 pages, everything."*

Why she's proud of it: *"All of my classmates at that time were like, 'oh, MNC is a good book' — because they were creating 4 to 5 books like this. I created only ONE book, and inside one page everything is there for networking. It was like an achievement."*

✅ **Her claim, and it's a useful bar to aim at:** *"No one is going to ask you something beyond this page or beyond this book — in networking and in Linux."*

She offered to share it with the class.

◆ ◆ ◆

04 🌐 Networking Kickoff

The resources

📚 *"There are 8 to 9 networking repos, one for each concept. I made them public."* (The seven linked at the top of this document are those repos.)

She polled the room first

🎤 SAID IN CLASS

"Tell me what kind of concepts you know in networking, so I get to know how much you know and what I need to teach you. Because I have 4–5 topics I need to teach today."

What the room already knew: **gateway**, **NAT**. Her read: *"Most of the things you know. You need a networking refresh."*

😄 CLASS MOMENT

And her honest assessment of the 5:15 PM energy: *"This class has no vibe. I'm not getting it — but I'm not seeing you all."* 😄

◆ ◆ ◆

05 🔢 IP Addresses — The Basics

What is an IP address?

🎤 SAID IN CLASS

She refused the textbook answer. *"Internet Protocol — I know, I know. I am not your professor to teach you in that manner. What do you mean by IP* address*?"*

Student: *"Unique address."*

✅ *"Unique address — perfect. A unique address given to devices. That's why we call it an IP address."*

🧠 PICTURE THIS

Picture this: an IP address is a **street address for a device**. Your house has one so post can reach you. Your laptop has one so packets can reach it. That's the whole concept.

How big is it?

Question	Answer
How many bits?	**32 bits**
Arranged how?	**4 groups of 8 bits** (octets) — `8 . 8 . 8 . 8`
So the range is…	`0.0.0.0` → `255.255.255.255`

🎮 Team quiz: how many ways to check your own IP?

Command	Note
`ip a`	the modern Linux way ⭐
`ip addr show`	the long form
`ifconfig`	the classic — needs `net-tools` installed on a fresh server
`hostname -i`	the sneaky one from Class 2

⚠️ WATCH OUT

The correction she made twice: `ipconfig` is the *Windows* command. On Linux/macOS it's `ifconfig` or `ip a`. The class kept mixing them up. *"There is only `ifconfig` command — `ipconfig` is not there."*

◆ ◆ ◆

06 🧱 IP Classes, Subnet Masks & Broadcast

🎮 How many classes are there?

🎤 SAID IN CLASS

Room: *"Three."* Her: *"I have five. I learned 5."*

✅ **There are 5: A, B, C, D, E.** Only A/B/C get taught normally, because D and E are reserved (multicast and experimental) and you never assign them to a laptop.

Class	First-octet range	Default subnet mask	Network bits	Host bits
A	1 – 126	`255.0.0.0`	8	24
B	128 – 191	`255.255.0.0`	16	16
C	192 – 223	`255.255.255.0`	24	8
D	224 – 239	*(multicast — reserved)*	—	—
E	240 – 255	*(experimental — reserved)*	—	—

📌 NOTE

Where's 127? Reserved for **loopback** (`127.0.0.1` = "this machine"). It belongs to no class, which is why Class A stops at 126.

📌 NOTE

Note on the recording: the exact numbers get garbled in this stretch (Telugu fragments over the top of her voice). The table above is the standard set, and matches the answers the class actually gave — *"Class A starting at 1"*, *"Class C has 192–223"* — plus her own `ip.md` scratch file in the course repo, which lists exactly these ranges.

Subnet mask — one job only

✅ LOCK THIS IN

Her definition, and honestly it's the best one you'll get:

"You need the subnet mask to identify which are the network bits and which are the host bits. That's it. Nothing else."

```
IP:    197 . 23 . 45 . 10
Mask:  255 .255 .255 .  0
       └─── NETWORK ──┘ └HOST┘
```

Broadcast address

🎤 SAID IN CLASS

Room: *"Last IP of the network."* Her: *"Close."*

✅ **Correct:** the broadcast address is the **final address** in the network/subnet range — the "shout to everyone on this street" address.

```
197.23.45.10  with mask 255.255.255.0  →  broadcast = 197.23.45.255
120.27.1.0    with mask 255.0.0.0      →  broadcast = 120.255.255.255
```

Counting hosts (from her own `ip.md`)

For a Class A address like `120.27.1.0/8`:

```
32 bits total
−  8 network bits
──────────────────
= 24 host bits

total addresses  = 2^24
USABLE hosts     = 2^24 − 2      ← minus network address, minus broadcast
```

✅ **LOCK THIS IN**

Always subtract 2. The first address in a range is the **network address** and the last is the **broadcast address** — neither can be given to a device.

Private IP ranges

The class discussed *"the range of private and public IP"* but the numbers didn't survive the audio. Here they are — these are fixed, standard, and worth memorising:

Range	CIDR	Class
`10.0.0.0` – `10.255.255.255`	`10.0.0.0/8`	A
`172.16.0.0` – `172.31.255.255`	`172.16.0.0/12`	B
`192.168.0.0` – `192.168.255.255`	`192.168.0.0/16`	C

💡 **GOOD TO KNOW**

Private vs public, plainly: a **private** IP is only meaningful **inside** your own network. A **public** IP is visible on the internet. Her `ip.md` lists the `10.x` range explicitly as the Class A private range.

◆ ◆ ◆

07 🔷 What Happens When You Open a Website

She walked this through live — and it maps almost exactly onto her `Networking` repo (§18.1), which does it layer by layer.

The short version

```
You click something on a website
        ↓
Request goes to DNS  →  resolves the name into an IP address
        ↓
(your own IP was already assigned by DHCP — see §9)
        ↓
Checked against CIDR ranges  →  routed
        ↓
Reaches the server, comes back, browser renders it
```

🧠 Why CIDR had to be invented

🧠 PICTURE THIS

Picture this. *"Previously, some different people created Class A, Class B, Class C. And then they got to know — okay, we have billions of devices. And now I don't have IP addresses left to assign. And they thought: okay, now what do we do?"*

The fix: CIDR — **C**lassless **I**nter-**D**omain **R**outing. Instead of being locked into three fixed sizes, you write `/24`, `/26`, `/21` and size the network to **exactly** the number of hosts you need.

```
Old way:  you're Class C → you get 256 addresses. Need 300? Too bad, take a Class B (65,536).
CIDR:     need ~500? Take a /23. Need ~60? Take a /26. Fits like a glove.
```

🎯 The genuinely mind-bending question she asked

🎯 INTERVIEW ALERT

"You are assigned an IP address. And some completely different organisation assigns the same IP to some other device. Two people have the same IP — have you ever imagined? Is it even possible? How?"

The answer:

✅ LOCK THIS IN

Because they're in two completely separate networks, each behind its own DNS and its own public IP.

```
scaler.com's network                  other-company.com's network
├── public IP: 168.x.x.x              ├── public IP: 233.x.x.x
└── inside it: 192.168.1.50   ↔       └── inside it: 192.168.1.50
    (same private IP!)                    (same private IP!)
```

"One is scaler.com, one is some other .com. It will resolve scaler.com inside their domain, which gives the main IP address for their domain."

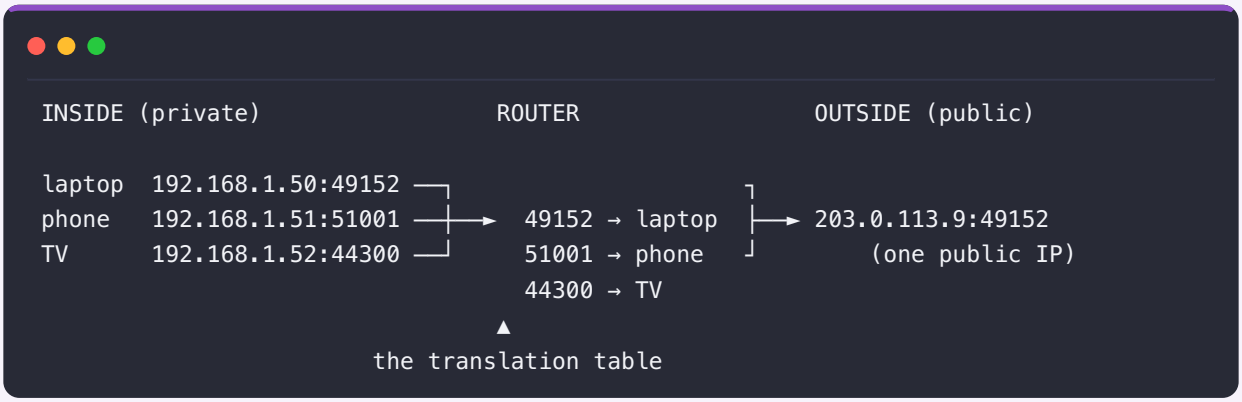
The rule: a private IP is unique **within its own network only** — never globally. The public IP + DNS is what makes each one uniquely reachable from outside.

🧠 PICTURE THIS

The piece that makes this work, and which the class already knew the name of: NAT.

Your whole house — laptop, phone, TV — shares **one** public IP. So when three devices all request Google at once, how does the reply find the right one?

NAT (Network Address Translation) is your router keeping a notebook:



On the way **out**, the router swaps your private IP for its public one and notes which port belongs to whom. On the way **back**, it reads the port, checks the notebook, and forwards the reply to the right device.

✅ **Two consequences worth knowing:** 1. **This is why the same `192.168.1.50` can exist in a million homes** — it never appears on the public internet. That's the answer to her "how can two people have the same IP?" question, mechanically. 2. **This is why you can't be reached from outside by default.** Nothing on the internet can start a conversation with your laptop, because there's no entry in the notebook yet. It's accidental security — and it's why port-forwarding exists when you *do* want to host something.

◆ ◆ ◆

08 🔷 Changing Your IP (and How Not To Lock Yourself Out)

The command

She walked the class to it one word at a time — *"IP address? Then? Add. Add, correct."*

```
ip address add 192.168.1.75 dev eth0
#              └─ the IP ─┘ └─ which interface
```

> 📌 **NOTE**
>
> **You must name the device.** *"You just need to mention the `dev` — I'm giving the IP address, then `dev`, then you need to mention your device. If you do this, then you're able to change it."*

If the IP is already taken:

```
Error: IP is already assigned to some other device
```

She showed her server had only **two** interfaces (`lo` and one real one), so adding a free IP just worked.

⚠️ The warning she repeated three times

> ⚠️ **WATCH OUT**
>
> **NEVER use `sudo` to change your IP address.**
>
> *"Never use sudo to change your IP address, by the way. Never."*
>
> **Why:** *"It will change your IP address permanently. And maybe you are not able to connect with your server."* On a remote box, that's an instant, unrecoverable lockout — you've moved the address you were connected on.
>
> ✅ **Her general rule:** *"We never use networking / IP address commands with `sudo`."*

🎯 `/etc/hosts` — a favourite interview question

> 🎯 **INTERVIEW ALERT**
>
> **The question:** *"Where are you getting all the host detail about your system?"* — note: **host detail**, not hostname.

```
cat /etc/hosts
```

What you see: your `localhost` entry, plus the network, nodes, and router mappings your device knows about locally.

> 💡 GOOD TO KNOW
>
> **When it's actually useful:** *"If I have a lot of devices — when I do `ip a` I can see two devices. But if I have 4 or 5 devices, then `cat /etc/hosts` shows me all the hosts I have."* It's your machine's own private mini-DNS.

◆ ◆ ◆

09 📡 DHCP — The DORA Handshake

She built this up as a fill-in-the-blanks Q&A with the class.

> 🧠 PICTURE THIS
>
> **Picture this.** You open your laptop in a café. It has **no IP address** — it can't ask anyone for one, because it doesn't have an address to be replied to. So it **shouts into the room**: *"Can anyone give me an address?!"* That shout is a **broadcast**, and DHCP is what answers it.

The four steps — DORA

Step	Direction	What it says
D — Discover	laptop 📣 **broadcasts**	*"Who can give me an IP address?"*
O — Offer	DHCP server → laptop	*"Here's `192.168.1.10`, plus the subnet mask, gateway and DNS."*
R — Request	laptop → server	*"Yes, I'd like to use `192.168.1.10`."*
A — Acknowledge	server → laptop	*"You're good to go. Lease: 24 hours."*

> 💡 GOOD TO KNOW
>
> **In your home, the DHCP server is your Wi-Fi router.** It's doing all four of these every time a device joins.

🎯 The best student question of the class

🎯 INTERVIEW ALERT

Student: *"Why do we need the Request and the Acknowledgement? Why can't the DHCP server just give us the information directly?"*

✅ **Her answer:** *"Because there is not one DHCP server — there are lots of them."*

```
Laptop broadcasts:  "I need an IP!"
                     ↓ heard by ALL 10 DHCP servers on the network
Server 3 offers:    "Take 192.168.1.10"
Laptop REQUESTS:    "I'm taking server 3's offer" ← broadcast, so everyone hears
                     ↓
Server 3 ACKs.  Servers 1,2,4–10 hear it and DON'T also assign an address.
```

"One DHCP server will acknowledge that it needs to give you the ID, and the other DHCP servers will not." Without the Request/Ack round-trip, ten servers would all hand you an address and you'd have a collision.

📌 NOTE

She noted that if there were genuinely only **one** DHCP server, the broadcast step would be less necessary — which is exactly the right instinct. The protocol is designed for the messy multi-server case.

😄 The exam-cram detour

This is where she went off on a tangent about her own college:

😄 CLASS MOMENT

"We were learning from this institute to pass our exam the night before. This is the material they were providing — every engineer used it. It's not our university, it's one institute that gives us this."

"If a networking paper was tomorrow — 10 PM after dinner we'd be like, okay, now we'll do it. We'd open the material for that subject. We know there are only six modules in the networking syllabus. We do not read it. That's it. We will pass. We have 70 marks. That's all — we're not even reading anything, we're just reading this PDF."

"That's why I liked this DHCP server configuration. While I was learning DevOps, I just put this in."

📌 NOTE

Sourcing note: a heavily garbled multilingual stretch (Portuguese + Telugu) follows here, where she continues on a DHCP-adjacent point and starts to set up a question about *why* the protocol is used. That specific passage didn't survive transcription clearly enough to reconstruct.

◆ ◆ ◆

10 🧱 Subnetting Refresher

The live example

Take a Class A address — she used something like `126.34.0.1` — and apply a mask.

```
126.34.0.1     with    255.0.0.0
└network┘└─── host ──┘
```

✅ LOCK THIS IN

Her one-sentence summary, again, because it's the whole topic: *"You need the subnet mask to identify which are the network bits and which are the host bits. That's it, nothing else."*

Reading a mask by counting the 255s

Mask	255s	Network bits	Class
`255.0.0.0`	1	8	A
`255.255.0.0`	2	16	B
`255.255.255.0`	3	24	C

💡 GOOD TO KNOW

The shortcut: each `255` = 8 network bits. Count the 255s, multiply by 8, and you have your CIDR number. Everything left over is host bits.

Full worked example — splitting a network into 4 subnets — is in §18.3.

◆ ◆ ◆

11 🔀 TCP vs UDP, and the OSI Layers

Run as a Team A/B/C quiz — one line per team.

The two protocols

	TCP	UDP
Full form	**T**ransmission **C**ontrol **P**rotocol	**U**ser **D**atagram **P**rotocol

	TCP	UDP
Acknowledges packets?	✅ **Yes**	❌ **No**
Speed	slower	⚡ **faster**
Reliability	reliable / secure	best-effort
Class's own words	*"transmitting the packets and acknowledging"*	*"transmitting the packets without acknowledgement"*

🧠 PICTURE THIS

Picture this: TCP is registered post — the courier gets a signature for every parcel, and re-sends anything that didn't arrive. **UDP is dropping flyers in letterboxes** — much faster, and nobody's checking whether they landed.

🎤 SAID IN CLASS

She asked why UDP is fast and got the answer straight back: *"Transmitting the packets without acknowledgement."* Correct — no waiting for a reply is the entire speed advantage.

A follow-up on whether UDP uses the same route or different routes each time was answered **"different route"** in class.

The OSI layer mapping

Protocol	OSI layer
HTTP / HTTPS	**Application** (Layer 7)
TCP / UDP	**Transport** (Layer 4)
IP / routing	**Network** (Layer 3)
MAC / switches	**Data Link** (Layer 2)
cables, Wi-Fi	**Physical** (Layer 1)

🎤 SAID IN CLASS

She wanted layer NAMES, not numbers: *"No, I don't want layer 1, 2, 3, 4. I need a layer name."*

🧠 Why HTTP lives in the Application layer

Her explanation, cleaned up:

🧠 PICTURE THIS

"A protocol is the intermediate between the user and the machine. Not between two devices — between you as a user and your laptop as a machine. So when you interact with the machine and open a website, the first layer that comes is the application layer, where we have HTTP or HTTPS."

🔍 **Slightly clarified, since this is easy to get muddled:** HTTP *is* also carried between two machines (your browser and the server). The useful part of her framing is that the Application layer is the layer **closest to you** — the one whose messages are about what *you* asked for ("GET me this page"), not about how bytes physically move. That's why HTTP sits at the top.

🔐 What makes HTTPS secure?

Answer	Who got it
It encrypts the data	first hand up
TLS	*"TLS — who told us TLS? Nice."*

✅ LOCK THIS IN

What TLS actually is: *"It's basically a certificate we call a TLS certificate, which does a handshake between two devices."*

🧠 Her analogy was literally a handshake — two parties confirming each other's identity before they start talking.

The two things to check whether a site is secure:

1. A **valid TLS certificate**
2. The **connection is HTTPS**

💡 GOOD TO KNOW

You've seen the failure mode: *"If you have an HTTP website, have you noticed a cross mark, 'not secure'? Some websites you open and you see 'if you want to open, security risk — then you can click and open.' Some security pop-up appears in Windows or Mac. It's because of HTTP."*

She demonstrated live by clicking the padlock on the attendance site → *"site information"* → showing the certificate.

◆ ◆ ◆

12 🎮 Ports Quiz

She fired these at the teams. The recording gets noisy here (several wrong guesses — one team said DNS was port "15"), so the correct table:

Service	Port
SSH	**22** ✅ *(the class got this one)*
HTTP	**80**
HTTPS	**443**
DNS	**53**
SMTP	**25**

😄 CLASS MOMENT

The DNS answer: *"Team C, DNS? 15? …Oh my."* 😄 (It's 53.)

🎯 The redirect question

🎯 INTERVIEW ALERT

"Normally, which port are we redirecting our application to? Let's say I run my application on port 8080. I want to redirect it. Which port?"

Answers came in: "80"… "8080"…

✅ *"80 is the correct answer. When you are redirecting your traffic, the default port is 80."*

🧠 PICTURE THIS

Why: browsers assume port 80 for `http://` and 443 for `https://`. If your app listens on 8080, every user has to type `example.com:8080`. Redirect 80 → 8080 and they just type `example.com`.

◆ ◆ ◆

13 🌐 Hands-On — The Networking Commands

This was the practical meat of the session.

The easy warm-ups

```
hostname        # this machine's name
whoami          # current username
ip a            # my IP address
```

`ping` — is it even alive?

```
ping google.com
```

What came back in her demo:

```
icmp_seq=1  ttl=117  time=9 ms
```

Field	Means
icmp_seq	which packet number this is
ttl	**T**ime **T**o **L**ive — how many more hops the packet may take
time	round-trip time

💡 GOOD TO KNOW

MAC vs IP — two addresses, and the reference material in §18 uses both.

	MAC address	IP address
Looks like	`f2:3a:73:cc:d9:6f`	`192.168.1.50`
Burned into	the **network card**, at the factory	assigned by **DHCP**, changes
Works	only on your **local** network	across the **whole internet**
OSI layer	**2** (Data Link) — switches	**3** (Network) — routers

🧠 **The postal analogy:** the **IP address** is the street address on the envelope — it's what routes the parcel across the country. The **MAC address** is *"hand it to the guy in the blue van"* — only meaningful for the very next hop, and it gets replaced at every depot.

✅ **Which is why the IP stays the same end-to-end, but the MAC changes at every single hop.** Your laptop's packet keeps Google's IP as its destination the whole way, while the MAC gets rewritten by every router it passes through.

✅ LOCK THIS IN

What `ping` actually sends: ICMP packets. *"One of the concepts we forgot — ICMP packets. It's nothing, just a simple file with some bytes in it. That's it. Zero-one data."*

🧠 **So the earlier TCP/UDP discussion connects:** ping isn't TCP or UDP — it's ICMP, a third thing, whose only job is "are you there, and how long did that take?"

Limiting the packet count:

```
ping -c 4 google.com     # send exactly 4 packets
ping -c 5 scaler.com     # she did 5 against Scaler
```

Her results: 4 transmitted, 4 received, **0% packet loss**. Then 5 to Scaler: 5 received, 0 lost.

😄 CLASS MOMENT

Live networking problems, in a networking class: her first ping didn't respond. *"I guess some internet issue is there, I'm sure. Or maybe some storage issue I had today. I don't know, but I'm not hitting."* Then Google worked. 😄

`route` / `ip route` — where does my traffic go?

```
route
ip route
```

💡 GOOD TO KNOW

The insight: *"You will see routes only for the devices you have. So if you do `ifconfig` and you see two devices, then you'll see only those routes."* Routes and interfaces come in pairs.

`nslookup` — ask the phonebook

```
nslookup google.com
```

🎤 SAID IN CLASS

"For which purpose are we using `nslookup` ?" → *"For DNS. To check the DNS."* ✅

What she showed: Google resolves to **many** different IPs.

💡 GOOD TO KNOW

And it explains the ping behaviour: *"Google has a lot of servers… so when I do `ping -c` google, they're sending the packets to different IPs, different servers."* One name, many machines — that's load balancing, visible from your terminal.

`curl` — fetch things

```
curl google.com          # dumps the raw HTML
curl -I google.com       # ⭐ just the HTTP HEADERS
```

What else `curl` does (she listed these):

- **Download a file** — including a zip from Google Drive
- **Install tools/libraries** — *"whenever you want to install Python, there's a specific script they mention"* (the classic `curl … | bash` installer)
- **Check whether a website responds**

The `-I` (headers) demo, and the class exercise:

> 🧪 **TRY IT YOURSELF**
>
> **She had everyone run `curl -I https://scaler.com` and compare one field: `Content-Length`.**
>
> Her result: **167**. Then round the room: *"Anyone got another response for content length?"* → *"167."* → *"167."* → *"Same? Okay. Good."*
>
> ✅ Everyone matched, which is the point of the exercise — the header is coming from the same server for everyone.

> 💡 **GOOD TO KNOW**
>
> **Why a DevOps/security person cares about headers:** *"When you're entering cyber security, or deep-diving into networking concepts, you want to check the header."* She then showed the browser equivalent: **DevTools → Network tab → reload → click the request → Headers / Payload**. Same information, two routes to it.
>
> 😄 *"I don't know, I never did this in Mac. I do this inspect on my company laptop."*

`ss -tuln` — what's listening on this box?

```
ss -tuln
```

Decoding the flags (she spelled this out):

Flag	Means
`ss`	list **s**ocket **s**tatistics — i.e. the ports
`-t`	**T**CP

Flag	Means
`-u`	**U**DP
`-l`	**l**istening only
`-n`	**n**umeric (show port numbers, not names)

What she saw: port **22** open for TCP, allowed for all.

🔗 …which explained an earlier EC2 mystery

✅ LOCK THIS IN

The connection she drew: when she created the EC2 instance back in Class 2, she allowed **SSH (22)** and **HTTP** — so `ss -tuln` now shows exactly those.

And the security-group setting finally makes sense:

Setting	Meaning
`0.0.0.0/0`	*"allowing the traffic from anywhere, any time, any place"*
My IP	only from your own address — *"then here I can see something else, as my IP address"*

`traceroute` — map the whole path

🎯 INTERVIEW ALERT

The setup question: *"From your laptop, when you're pinging Google — do you know what the path is between you and Google?"*

Room: *"Switches, routers…"* ✅ Right idea. Each device in between is a **hop**.

```
traceroute google.com      # install it first if it's missing
```

What she saw: roughly **64 hops** being probed between her machine and google.com.

🎤 SAID IN CLASS

"From my address it will go to 244.5.8.217, then again 246, then the next server, then the next server, and then the main server will redirect my request to google.com."

💡 **When a hop doesn't answer** you see stars/blanks: *"When there is no route you'll see something like this — it is searching, basically, from line 12 to line 16. Searching for the route to get to google.com. When it gets the route it will do it."*

✅ **The mental model she landed on:** *"Whenever I'm doing a ping, these things are running in the background. These are the things that we can see running in the background."* traceroute makes the invisible middle visible.

`telnet` — is that specific port open?

```
telnet google.com 80
```

🎤 SAID IN CLASS

"Why are we using the telnet command? To check the connectivity of a service — or the port connectivity of a service inside a server." ✅

What happens under the hood, in her words:

```
telnet google.com 80
  → google.com has its own DNS  →  resolves to an IP
  → Google runs a public service on port 80
  → telnet checks: is that service actually running?
  → "Connected to google.com"  ✅
```

Then she had the class try Scaler. Results were mixed — some connected, some didn't. *"Who is not connected? Not connected because of an issue — that's the only thing."* Left as an open troubleshooting point.

✅ LOCK THIS IN

The three-tool debugging ladder from Class 1, now with hands-on experience:

Step	Command	Answers
1	`ping host`	is the machine reachable at all?
2	`traceroute host`	if not — where is it dying?
3	`telnet host port`	machine is up — but is *this service* up?

◆ ◆ ◆

14 📝 Homework

#	Task
1	🔗 `nslookup` **task:** find the DNS/IP info for **scaler.com** and **google.com**, put the commands in a `.sh` file in a **new GitHub repo**, add a **screenshot** of the output, and share the link
2	📄 Create an `.md` file listing **all the commands/concepts from today** (ping and friends)
3	📖 Read the `How-DHCP-Works` repo — complete DHCP flow (§18.2)
4	📖 Go through `IP-quest` — the interview question set (§18.4)
5	🔀 Anyone who hasn't yet: **raise the PR** on the DevOps Heroes repo
6	🐳 **Install Docker** before the Docker session (Docker Desktop if you're on Windows)
7	🌿 **Install Git Bash** before tomorrow's Git session

◆ ◆ ◆

15 👋 Wrap-Up

Her own recap

🎤 SAID IN CLASS

"In today's session we covered ports, we covered IP, TCP, UDP, DHCP, IP-quest, and hostname."

Repo organisation

All networking resources go into the same **DevOps Heroes** repo, in a session folder.

😄 CLASS MOMENT

She let the class vote on the folder name — *"Networking"* or *"session 4"*? The class said **Networking**. (In the actual repo it ended up as `session4-networking` — both. 🙂)

🎯 How networking is ACTUALLY tested

🎯 INTERVIEW ALERT

This is the most useful thing she said all class:

"In the networking session there are only 2 to 3 to 4 commands. It's about the knowledge you have and how you can use it. If you know TCP, UDP, IPv4, IPv6, and a few commands like telnet, IP address, how to find IP — these kinds of questions will come."

And what they will NOT ask: *"They will not ask you to write out the complete IP diagram, how many bits are there. They are not asking that."*

What they WILL ask — and these come straight from her `IP-quest` repo:

The question	Where it shows up
"Subnet `192.168.0.0/24` — what's the highest possible host address?"	OA round
"Subnet mask `255.255.252.0` — how many host bits?"	🔥 *"This is the favourite question. In every interview I got this type of question."*
"How many hosts are possible? 2^8? 2^5?"	practical / technical round 2 or 3

😄 **And the confession:** *"The last one I got in a Flipkart interview — and I got it wrong, by the way."* 😄

What's next

Session	Topic
Next	**Git & GitHub**, deeper — **rebase**, **merge conflicts**. *"It will be a basic session, but after that we take it to intermediate."*
After that	**Docker** — the first properly "intermediate" topic

Prep for Docker:

📌 NOTE

"You need to install Docker. If you have Windows, Docker host is there — I just installed it, Docker Desktop is there, Docker host is there. And if you have a server then you can install it on the server. But make sure to install Docker."

Prep for Git:

😄 **CLASS MOMENT**

"For tomorrow's session — Git Bash. You have it? You know it? Then how are you pushing the code? How are you pushing the code without Git Bash? Are you directly uploading the directory to GitHub, or are you pushing with `git add`, `git commit`?" 😄

Closed with attendance by roll number (Section A), and:

✅ **LOCK THIS IN**

Her parting line, which is the honest summary of the whole class: *"See — networking you just need to know the concept. In Linux you need to do the hands-on. There are two different things."*

(The recording cuts off mid-sentence during the roll call, same as Class 3.)

◆ ◆ ◆

16 📔 Cheat-Sheet — Commands & Concepts

Commands

Command	Does
`hostname`	machine's hostname
`whoami`	current username
`ip a`	show IP addresses (Linux/macOS)
`ifconfig`	show/configure interfaces — needs `net-tools`. **Not** `ipconfig` (that's Windows)
`ip address add <ip> dev <iface>`	manually assign an IP — ⚠️ **never with** `sudo`
`cat /etc/hosts`	🎯 local host→IP mappings (interview question)
`ping <host>`	is it reachable? sends **ICMP** packets
`ping -c <n> <host>`	send exactly *n* packets
`route` / `ip route`	routing table
`nslookup <domain>`	DNS lookup
`curl <url>`	fetch a page's HTML / download / install
`curl -I <url>`	fetch just the **headers**
`ss -tuln`	listening ports — **t**cp, **u**dp, **l**istening, **n**umeric

Command	Does
`traceroute <host>`	hop-by-hop path to a destination
`telnet <host> <port>`	is this specific **port** reachable?

Concepts

Term	Means
IP address	unique 32-bit address for a device — 4 × 8-bit octets
Public IP	visible from the internet
Private IP	valid only inside a local network — `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`
Class A/B/C/D/E	the 5 IPv4 classes; D = multicast, E = experimental (both reserved)
`127.0.0.1`	loopback — "this machine"
Subnet mask	says which bits are **network** and which are **host**. That's all.
Network address	first address in a range — not assignable
Broadcast address	last address in a range — not assignable
Usable hosts	`2^host_bits − 2`
CIDR (`/24` **)**	classless notation — size the network to what you need
Gateway	the way out of your network (usually your router)
NAT	translates private IPs → public IP on the way out — see §7
DNS	name → IP; the internet's phonebook
DHCP / DORA	**D**iscover → **O**ffer → **R**equest → **A**cknowledge
Lease time	how long a DHCP-assigned IP stays yours
TCP	reliable, **acknowledges** packets
UDP	fast, **no** acknowledgement
ICMP	what `ping` uses — "are you there?"
TTL	Time To Live — hop budget for a packet
Hop	each router a packet passes through
TLS certificate	powers the HTTPS encrypted handshake
Switch	Layer 2 — forwards frames by **MAC** address, inside the LAN
Bridge	Layer 2 — joins two LAN segments
Router	Layer 3 — forwards packets **between** networks; usually also the DHCP server

Term	Means
OSI: Application	HTTP, HTTPS, DHCP
OSI: Transport	TCP, UDP
OSI: Network	IP, routing
OSI: Data Link	MAC, switches
OSI: Physical	cables, Wi-Fi

Ports

Port	Service
22	SSH
25	SMTP
53	DNS
80	HTTP
443	HTTPS
8080	common app default — usually redirected to 80
`0.0.0.0/0`	security-group source meaning "**from anywhere**"

◆ ◆ ◆

17 🔷 The Interview Question Set (from `IP-quest`)

These are her actual collected interview questions. **Work through them — they're the ones that get asked.**

Q1 — In subnet `192.168.0.0/24`, what is the highest possible host address?

```
/24  →  mask 255.255.255.0  →  first 24 bits = network, last 8 = host

Network address    192.168.0.0     ← reserved, can't assign
Usable range       192.168.0.1  →  192.168.0.254
Broadcast address  192.168.0.255   ← reserved, can't assign
```

✅ LOCK THIS IN

Answer: `192.168.0.254` — the one just before the broadcast address.

Q2 — In subnet mask `255.255.252.0`, how many host bits are there? 🔥

🔥 HARD MODE

"This is a good question — in every interview I got this type of question."

Step 1 — convert to binary:

```
255  →  11111111
255  →  11111111
252  →  11111100
  0  →  00000000

255.255.252.0 = 11111111.11111111.11111100.00000000
```

Step 2 — count the 1s (network bits):

```
8 + 8 + 6 + 0 = 22
```

Step 3 — subtract from 32:

```
32 - 22 = 10
```

✅ LOCK THIS IN

Answer: 10 host bits.

Q3 — How many total host addresses with mask `255.255.248.0`?

```
255  →  11111111    (8)
255  →  11111111    (8)
248  →  11111000    (5)
  0  →  00000000    (0)
                    ──
network bits         21
host bits    = 32 - 21 = 11
```

```
total host addresses  = 2^11 = 2048
usable                = 2048 - 2 = 2046
```

✅ LOCK THIS IN

Answer: 2048 total, 2046 usable (minus network + broadcast).

Q4 — A `/21` subnet — how many usable IP addresses?

```
host bits = 32 - 21 = 11
total     = 2^11    = 2048
usable    = 2048 - 2 = 2046
```

✅ LOCK THIS IN

Answer: 2046.

🧱 How to convert a number to binary in your head

You need this for Q2 and Q3, and it's much easier than it looks. **There are only 9 numbers that ever appear in a subnet mask.** Learn the table and you never convert anything again:

Decimal	Binary	1-bits
255	`11111111`	8
254	`11111110`	7
252	`11111100`	6
248	`11111000`	5
240	`11110000`	4
224	`11100000`	3
192	`11000000`	2
128	`10000000`	1
0	`00000000`	0

🧠 PICTURE THIS

Why only those nine? A subnet mask is always **all the 1s on the left, all the 0s on the right** — never mixed. So each octet can only be one of nine patterns. `11011000` is not a legal mask.

If you'd rather work it out than memorise it, use place values. Each bit position is worth double the one to its right:

```
128   64   32   16    8    4    2    1
 1    1    1    1     1    0    0    0    =  128+64+32+16+8  =  248   (5 ones)
```

Just take the number and greedily subtract from the left: `248 − 128 = 120`, `120 − 64 = 56`, `56 − 32 = 24`, `24 − 16 = 8`, `8 − 8 = 0` → five subtractions → **five 1-bits**.

✅ LOCK THIS IN

Shortcut for the exam: you don't need the binary at all — you need the **count of 1-bits**, which is the CIDR number. `255.255.252.0` → `8 + 8 + 6 + 0` = **/22**. Then `32 − 22 = 10` host bits. Done in five seconds.

🧱 The universal method

✅ LOCK THIS IN

Every one of these questions is the same three steps:

```
1. host bits    = 32 − (CIDR number)          [or: 32 − count of 1s in the mask]
2. total hosts  = 2 ^ host_bits
3. usable hosts = total − 2                    [network address + broadcast address]
```

Memorise the powers of two and you can do these in your head:

Host bits	Total	Usable
8 (`/24`)	256	254
9 (`/23`)	512	510
10 (`/22`)	1024	1022
11 (`/21`)	2048	2046
12 (`/20`)	4096	4094
6 (`/26`)	64	62

◆ ◆ ◆

18 🌐 Reference Material — Her Networking Repos

She shared these as the session's study material. Reorganised here for readability; all the technical content is preserved.

18.1 The full journey — laptop → Google → back

Source: Nency-Ravaliya/Networking

Step 1 — your laptop gets an IP (DHCP)

#	Message	Layer	What's in it	What the switch does
1	**DHCP Discover**	Application (L7)	broadcast to `255.255.255.255` — it doesn't know the server's IP yet	forwards to **all** connected devices; only the router (the DHCP server) replies
2	**DHCP Offer**	Application	proposed IP (`192.168.1.50`), subnet mask (`255.255.255.0`), default gateway (`192.168.1.1`)	forwards **only to your laptop**, using its MAC address from the switch's table
3	**DHCP Request**	Application	*"I'll take the offered IP"*	—
4	**DHCP Acknowledgment**	Application	confirms IP, gateway, subnet mask, **DNS server**	—

✅ **LOCK THIS IN**

Now your laptop has an IP and knows how to route traffic through the router (the **default gateway**).

Step 2 — connecting to google.com

Layers 1–2 · Physical & Data Link — the switch

- Your request leaves as an **Ethernet frame**, addressed with your laptop's **MAC address** and the MAC of the next hop (your router).
- The switch checks its **MAC address table** to pick which port to forward it out of.

Layer 3 · Network — the router

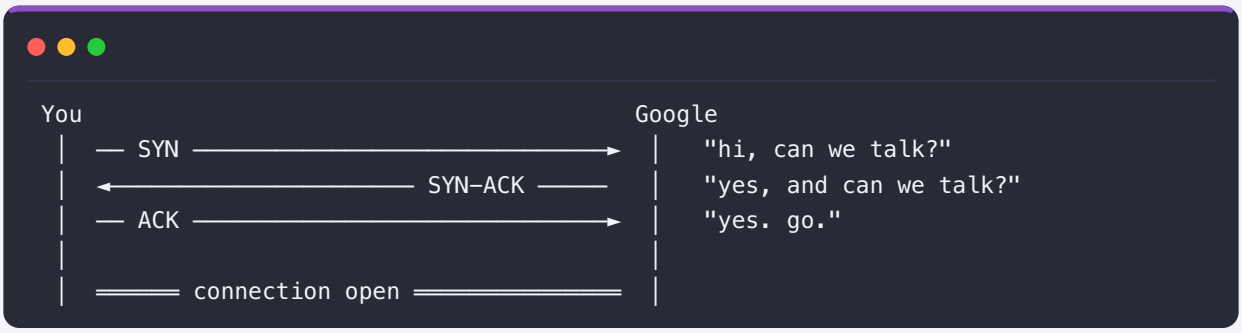
- The router unwraps the frame and reads the **IP packet** inside:
- **Source IP:** your laptop, e.g. `192.168.1.50`
- **Destination IP:** one of Google's IPs, e.g. `142.250.190.14`
- The router has **two** addresses: a local one (`192.168.1.1`) and a **public-facing** one from your ISP.
- It consults its **routing table** and forwards the packet out of its **WAN** interface toward the ISP.

Layer 4 · Transport — TCP

- Inside the IP packet sits the **TCP segment**:
- **Source port:** a random high port from your laptop, e.g. `49152`
- **Destination port:** `80` (HTTP) or `443` (HTTPS)
- The connection is established with the **TCP three-way handshake: SYN → SYN-ACK → ACK.**

🧠 PICTURE THIS

The three-way handshake, in three lines of dialogue:



✅ **Why three and not two:** both sides need to confirm they can *hear* the other. Two messages only proves one direction works. Three proves both — and this is the "acknowledgement" that makes TCP reliable and UDP fast (§11).

💡 **You've already met the failure mode.** When `telnet google.com 80` prints *"Connected"*, this handshake just completed. When it hangs, the SYN went out and no SYN-ACK ever came back.

Layer 3 again · hops across the internet

- The ISP passes the packet through a series of **hops**. Each hop is a router that reads the destination IP and forwards it closer.
- 💡 **This is exactly what `traceroute` shows you.**

Step 3 — Google responds

- Google's server sends back the HTML — now with **Google's IP as source** and **your IP as destination**.
- Same TCP ports keep the connection open (e.g. 443).
- It travels back through the same hops → your router → the switch → your laptop's MAC address.
- Your **browser** renders the page. Done.

The four devices, summarised

Device	Layer	Job
Switch	2	forwards frames by **MAC** address inside the LAN. Does **not** assign IPs.
Bridge	2	connects two separate LAN **segments**
Router	3	forwards packets **between networks**; has a local IP **and** a public IP; usually **is** the DHCP server

Device	Layer	Job
Hop	—	any intermediary router between you and the destination

What's actually in the request and response

	Request → Google	Response ← Google
Source IP	your laptop (`192.168.1.50`)	Google's server (`142.250.190.14`)
Destination IP	Google's server	your laptop
Transport	ports `49152 → 443`	the reply on the same connection
Data	your browser's HTTP request	the HTML for the page

◆ ◆ ◆

18.2 DHCP in detail

Source: Nency-Ravaliya/How-DHCP-Works

✅ **LOCK THIS IN**

What DHCP is: a protocol that **automatically** assigns IP addresses, subnet masks, gateways and DNS settings to devices when they join a network — so nobody has to configure each device by hand.

The four messages, with real contents

1 · DHCP Discover *(laptop → router)* — **Layer 2 broadcast**, because the laptop has no IP yet, only a MAC address.

```
DHCP Discover: Who can give me an IP address?
Source:      Laptop's MAC address
Destination: Broadcast (FF:FF:FF:FF:FF:FF)
```

2 · DHCP Offer *(router → laptop)* — **Layer 3**, still broadcast on the local network.

```
DHCP Offer: Here's an IP address (192.168.1.10) for you.
Offered IP:   192.168.1.10
Subnet Mask:  255.255.255.0
Gateway:      192.168.1.1
DNS:          8.8.8.8        (Google's DNS)
```

3 · DHCP Request *(laptop → router)* — **Layer 3**, also broadcast, so every other DHCP server hears that this client is taken.

```
DHCP Request: I'd like to use 192.168.1.10.
Source:       Laptop's MAC address
Requested IP: 192.168.1.10
```

4 · DHCP Acknowledgement *(router → laptop)* — **Layer 3**. Now the laptop is on the network.

```
DHCP ACK: You're good to go with 192.168.1.10.
Lease Time: 24 hours
DNS:        8.8.8.8
```

The five things DHCP hands you

Parameter	Example	What it's for
IP address	`192.168.1.10`	your device's local address
Subnet mask	`255.255.255.0`	defines the size of your local network
Default gateway	`192.168.1.1`	your router — your way **out** to the internet
DNS server	`8.8.8.8`	translates `google.com` → an IP
Lease time	24 hours	after this you must renew, or get a new IP

DHCP across the OSI layers

Layer	Role in DHCP
L1 Physical	the requests travel over Ethernet/Wi-Fi
L2 Data Link	at first your laptop only knows its **MAC** address
L3 Network	DHCP assigns the IP, subnet mask and gateway → now L3 works
L7 Application	DHCP itself is an **application-layer** protocol

Three concepts worth naming

Concept	Meaning
Dynamic IP assignment	IPs come from a **pool** (e.g. `192.168.1.2` – `192.168.1.254`); nothing is configured by hand
Lease time	IPs are **rented**, not owned — renew or lose it

Concept	Meaning
Broadcasting	Discover and Request are broadcasts because the device doesn't know its own IP or the server's

◆ ◆ ◆

18.3 Subnetting — a full worked example

Source: Nency-Ravaliya/Subnetting

🧠 PICTURE THIS

Why subnet at all? *"Breaking a large network into smaller sub-networks helps organise networks, improve security, and optimise performance."* One giant flat network is a security and traffic nightmare; four smaller ones are manageable.

The scenario: split `192.168.1.0/24` into 4 subnets

Step 1 — how many subnets do you need? → 4.

Step 2 — borrow bits from the host part.

```
4 subnets = 2^2  →  borrow 2 bits

/24 + 2  =  /26        →  new mask: 255.255.255.192
```

Step 3 — how many addresses per subnet?

```
host bits = 32 − 26 = 6
total     = 2^6     = 64 addresses per subnet
```

Step 4 — how many are actually usable?

```
64
− 1  network address   (first in the subnet)
− 1  broadcast address (last in the subnet)
────
62 usable
```

Step 5 — the four subnets:

#	Subnet	Network address	Usable range	Broadcast
1	`192.168.1.0/26`	`192.168.1.0`	`.1` → `.62`	`192.168.1.63`
2	`192.168.1.64/26`	`192.168.1.64`	`.65` → `.126`	`192.168.1.127`
3	`192.168.1.128/26`	`192.168.1.128`	`.129` → `.190`	`192.168.1.191`
4	`192.168.1.192/26`	`192.168.1.192`	`.193` → `.254`	`192.168.1.255`

Before and after

	Original `/24`	After splitting into `/26`
Subnets	1	**4**
Total addresses	256	64 each
Usable addresses	254	**62 each** (248 total)

> ⚠️ **WATCH OUT**
>
> **Notice the cost:** you had 254 usable addresses, and now you have 248. Every extra subnet burns two more addresses on its own network + broadcast pair. Splitting isn't free.

◆ ◆ ◆

18.4 IPFIX / NetFlow / NTP — watching the traffic

Source: Nency-Ravaliya/IPFIX-NETFLOW-NTP

> 📌 **NOTE**
>
> Shared as this session's material, though not walked through line-by-line in the captured audio.

What they are

IPFIX (IP Flow Information Export) and **NetFlow** monitor and analyse network traffic by capturing details of **network flows**.

> 🧠 **PICTURE THIS**
>
> **Picture this:** it's like tracking every "conversation" happening on a network. Each conversation is a **flow**, and IPFIX/NetFlow logs who talked to whom, how much data moved, and for how long.

What counts as a "flow"

A stream of packets sharing the same:

- **source IP** · **destination IP**
- **source port** · **destination port**

• **protocol** (TCP / UDP)

> 💡 **GOOD TO KNOW**
>
> Open one web page → that's **one flow**.

Why anyone bothers

Reason	What it buys you
Traffic analysis	who's talking to whom, what kind of traffic, how much data
Security	spot unauthorised access attempts, unexpected large transfers
Performance	find bottlenecks, high latency, packet loss

How it works — three steps

```
1. CAPTURE   routers / switches / firewalls record each flow's
             IPs, ports, protocols, timestamps, byte counts
                          ↓
2. EXPORT    the device sends that data to a COLLECTOR
                          ↓
3. ANALYSE   the collector organises it into readable dashboards
```

🏢 The office scenario (50 employees)

You're the IT manager. Three people are doing three things:

Person	Activity	The flow captured
John	watching YouTube	John's IP ↔ YouTube's server — 30 min, **500 MB**
Sarah	emailing a client	Sarah's IP ↔ Gmail — 10 emails, very little data
Alex	downloading from Dropbox	Alex's IP ↔ Dropbox — **2 GB** file

The router sees all of it and exports it to a **flow collector**, which tells you exactly who's using the bandwidth and whether anything looks suspicious.

⏰ Why NTP matters here

NTP (Network Time Protocol) synchronises the clocks of every device on a network.

> ⚠️ **WATCH OUT**
>
> **The problem without it:** the router says John started YouTube at **10:00 AM**. The flow collector says **10:05 AM**. Same event, two timestamps — and now your incident timeline is fiction.

How NTP works:

1. An **NTP server** provides the reference time.
2. Every device (routers, servers, collectors) syncs its internal clock to it.
3. All timestamps now line up, so flows can be **correlated across devices**.

> 🧠 **PICTURE THIS**
>
> **The analogy from the repo:** *"It's like setting all the clocks in your office to the exact same time so everyone knows when meetings start. Without synchronised clocks, meetings could start at different times depending on which clock you look at."*

The full picture

```
Laptop → "google.com"
    ↓
Router  · logs the flow (your private IP → Google's IP)
        · NATs your private IP to the public IP
        · forwards the request
    ↓
Flow collector  ← receives the exported flow data
    ↑
NTP server  → keeps the router and collector on the same clock
    ↓
Network admin analyses: patterns, threats, performance
```

Component	Job
Router	sees all traffic, records flows, exports via NetFlow/IPFIX
Flow collector	receives and analyses the flow data
NTP	keeps every device's clock identical
Flow data	IPs, timestamps, bytes, source & destination

◆ ◆ ◆

18.5 Her other two networking repos

Repo	Topic
Network-Troubleshooting	debugging a broken network, hands-on
OSI-Network-devices	which device lives at which OSI layer

All of them are collected in her networking list.

◆ ◆ ◆

19 🗒️ Notes on the Transcript Itself

- **Instructor name:** the header credits *"Ritesh Prajapati"* — that's the **Scaler++ Chrome extension's developer**, not the teacher. The instructor is **Nensi Ravaliya** (@Nency-Ravaliya), DevSecOps Engineer at Standard Chartered.
- **This is the noisiest recording of the course so far.** Alongside English, it contains long stretches of **Telugu, Hindi, Portuguese and Ukrainian** — transcription/dubbing artifacts, not speech. Several minutes of the IP-classes and ports discussion are buried under them.
- **Where meaning was recoverable**, it's folded into the sections above. **Where it wasn't** — the DHCP-adjacent passage in §9 and parts of the ports quiz in §12 — that's stated plainly rather than guessed at. Numbers that came through partially (IP class ranges, port numbers) are given here in their standard, correct form, cross-checked against her own `ip.md` in the course repo.
- `Content-Length: 167` **vs** `567` **:** the transcript contains one "567" early in the exchange, then the class converges on **167** repeatedly. 167 is the value.
- **The script filenames** *"value.sh"* and *"wildhook.sh"* are both ASR renderings of `while_loop.sh` from Class 3 — same behaviour, same file.
- Garbled terms corrected: **"IP config"** → `ifconfig` / `ip a`, **"call command"** → `curl`, **"call-i"** → `curl -I`, **"NSLOOKUP / ns.scaler.com"** → `nslookup`, **"delnet"** → `telnet`, **"test out / trashroot / press route"** → `traceroute`, **"hopes / groups / rounds"** → **hops/routes**, **"DXAP / DNCP / DCP"** → **DHCP**, **"CITF / see idea"** → **CIDR**, **"submit mask / subject mask"** → **subnet mask**, **"MNC is a good book"** → **an offhand comparison to a thick reference book**, **"GlassPay"** → **an unclear tool name in her cheat-sheet list**, **"4074 microseconds"** → **most likely milliseconds in the ping summary**.